



Universidad Católica de la Santísima Concepción
Facultad de Ingeniería
Sistemas Digitales

Tarea 3
Síntesis y diseño de circuitos de mediana complejidad

Profesora: Yaime Fernández
Integrantes: Castillo Benjamín
Cegarra Ramón
Parra Paola
Ramos Edinson
Rossel Francisco
Sanhueza Angelo

Julio 2025, Concepción

Índice

Introducción.....	3
Objetivo.....	3
Materiales utilizados.....	3
Definición del problema.....	3
Funcionamiento general.....	4
Resultados observados.....	5
Aportes de cada integrante.....	5
Tabla 1. Principales aportes de cada integrante.....	6
Principales dificultades y desafíos.....	6
Conclusiones.....	6
Anexos.....	7
1. Montaje.....	7
2. Capturas del display.....	7
Figura 1. Display módulo 1.....	7
Figura 2. Display módulo 2.....	7
Figura 3. Display módulo 3.....	8
Figura 4. Display módulo 4.....	8
3. Código fuente.....	8
Linkografía.....	14

Introducción

En el presente informe se expone la implementación de un sistema de detección de luminosidad utilizando la plataforma Arduino. Arduino es una herramienta interactiva de software y hardware que permite a personas con distintos niveles de experiencia desarrollar proyectos electrónicos tanto simples como complejos.

Para este trabajo se empleó la placa Arduino MKR WiFi 1010 junto con el módulo MKR IoT Carrier, el cual incorpora un microcontrolador, botones, módulo de comunicación inalámbrica, luces RGB, pantalla integrada, sensores entre otros componentes. En particular, se hizo uso del sensor de luminosidad integrado en el dispositivo para medir los niveles de luz en el entorno.

A lo largo del informe, se detallará el funcionamiento del sistema propuesto, se explicará la lógica detrás del código desarrollado y se analizarán los desafíos técnicos encontrados durante la implementación.

Objetivo

Diseñar una solución basada en microcontrolador “Arduino MKR WiFi 1010 + MKR IoT Carrier” que permita detectar intensidad de luminosidad en el entorno, y clasificar estos niveles en rangos definidos para activar señales visuales (leds RGB) y acústicas (buzzer) de acuerdo con la intensidad detectada.

Materiales utilizados

1. Placa Arduino MKR Wifi 1010
2. MKR Iot Carrier
3. Cable micro USB
4. IDE Arduino
5. Sensor de luminosidad (integrado)
6. Leds RGB (integrado)
7. Buzzer (integrado)

Definición del problema

El problema a solucionar está definido por el siguiente enunciado:

Diseñe un módulo basado en el microcontrolador Arduino MKR WiFi 1010 + MKR IoT Carrier que cumpla con las siguientes características:

Debe medir la iluminación (L) que experimenta el módulo y mostrarla en pantalla.

Definir tres valores de luminosidad $L1 < L2 < L3$.

o Si $L \leq L1$, mantenga apagados los leds.

- o Si $L1 < L \leq L2$, encienda 1 led RGB.
- o Si $L2 < L \leq L3$, encienda 3 leds RGB.
- o Si $L > L3$, encienda los 5 leds RGB y active una alarma usando la bocina. La bocina debe permanecer activa por 3 segundos, luego de entrar en el estado $L > L3$, y luego desactivarse.

Los valores de luminosidad escogidos deben ser tales que permitan observar los 4 estados en el laboratorio, sin dañar el módulo.

Es necesario llevar la cuenta del tiempo que el módulo ha experimentado el estado $L \leq L1$ y mostrarlo en pantalla.

Para abordar el problema, el primer paso fue familiarizarse con el software de programación utilizado por Arduino. Para ello, se realizaron pruebas utilizando códigos proporcionados por la docente, los cuales incluían el uso de sensores de luminosidad, humedad, así como la utilización de la pantalla del dispositivo. Luego, una vez comprendido el funcionamiento básico, se dio inicio al desarrollo de un nuevo código fuente para solucionar el problema propuesto.

Funcionamiento general

La solución implementada en este proyecto tiene como finalidad medir en tiempo real el nivel de luminosidad del entorno, utilizando el sensor de luminosidad integrado en el MKR IoT Carrier. Con base en ese valor, clasifica la intensidad en uno de cuatro rangos definidos por parámetros, y ejecuta acciones específicas dependiendo del caso.

([Ver anexo, Código fuente](#))

1. Variables de control

A continuación, se describen las variables utilizadas en el programa para gestionar el flujo del sistema y el comportamiento de cada módulo.

([Ver anexo, Bloque de código 1](#))

light: Variable entera que almacena el valor actual de luminosidad medido por el sensor.

L1, L2, L3: Variables enteras que definen los rangos de clasificación de la luminosidad.

r, g, b: variables enteras que almacenan los valores de color (rojo, verde y azul) usados para modificar el fondo de pantalla según el módulo activo.

duracionEnModulo: Variable unsigned long que almacena el tiempo (en milisegundos) desde que el sistema ingresó al Módulo 1.

tiempo: Variable entera para almacenar tiempo de inicio de Módulo 1.

estarEnModulo: bandera booleana que indica si el sistema ya se encuentra en Módulo 1, con el fin de registrar una sola vez el tiempo de entrada.

sonarBuzzer: bandera booleana utilizada para controlar la activación del buzzer, evitando que se repita mientras se mantenga en Módulo 4.

2. Flujo de funcionamiento

Lectura del sensor:

En el bucle principal (loop()), se espera a que el sensor de luz tenga un nuevo dato disponible. Luego, se llama a readColor() para obtener el valor de luminosidad almacenado en la variable light.

Clasificación del nivel de luz e impresión por pantalla:

La función Clasificar() analiza el valor de light y lo compara con los parámetros definidos L1, L2, L3 para determinar el estado actual de luminosidad ([Ver anexo, Bloque de código 2](#)). Cada rango activa un módulo correspondiente:

Módulo 1 ($\text{light} \leq L1$)

Se considera una condición de baja luminosidad. Todos los leds se apagan.
Se muestra un fondo azul en pantalla con el valor actual de luminosidad.
Tiempo en segundos que el sistema ha permanecido en esta condición.
([Ver anexo, Figura 1](#))

Módulo 2 ($L1 < \text{light} \leq L2$)

Representa una luminosidad media-baja. Se enciende solo un led en color rojo.
Se muestra un fondo verde en pantalla con el valor actual de luminosidad.
([Ver anexo, Figura 2](#))

Módulo 3 ($L2 < \text{light} \leq L3$):

Representa una luminosidad media-alta. Se encienden tres LEDs en color azul, distribuidos alternadamente. Se muestra un fondo magenta en pantalla con el valor de luminosidad.
([Ver anexo, Figura 3](#))

Módulo 4 ($\text{light} > L3$):

Se considera una condición de luminosidad crítica o máxima. Se encienden todos los leds en color blanco. Se activa una alarma (buzzer) que suena durante 3 segundos. Para evitar que la alarma se repita mientras se mantiene en este estado, se utiliza una bandera sonarBuzzer que permite sonar solo una vez hasta que cambie el estado.
([Ver anexo, Figura 4](#))

Resultados observados

1. El sistema responde correctamente a variaciones de luz ambiental.
2. El display muestra la información de manera clara.

3. Los leds reflejan visualmente el nivel detectado.
4. El buzzer se activa solamente al ingresar en niveles críticos y cumple con el tiempo programado.

Aportes de cada integrante

Integrante	Aporte principal
Benjamín Castillo	Colaboró en la creación, estructuración del código con la codificación de la función Clasificar() . Además, participó en la redacción del informe y depuración de código.
Ramón Cegarra	Participó en la elaboración del código investigando el uso de la función millis() , así como en la creación y diseño de la presentación (PPT) del proyecto.
Paola Parra	Desempeñó el rol de líder del grupo. Fue responsable del cuidado y resguardo del material de trabajo. Participó en la creación, estructuración, depuración y testeo del código, así como en la validación de las funcionalidades del dispositivo. Además, contribuyó en la redacción del informe.
Edinson Ramos	Participó en la estructuración y depuración del código. Asimismo, contribuyó en la redacción del informe y está a cargo del diseño y desarrollo de la presentación (PPT) del proyecto.
Francisco Rossel	Realizó aportes en la etapa final del informe, especialmente en la formulación de las conclusiones del proyecto.
Angelo Sanhueza	Aportó en la creación, estructuración y depuración del código. Asimismo, contribuyó en la redacción del informe y desarrollo de la presentación (PPT) del proyecto.

Tabla 1. Principales aportes de cada integrante.

Principales dificultades y desafíos

Una de las principales dificultades durante el desarrollo del programa fue establecer una comunicación efectiva con la placa Arduino, ya que inicialmente ningún equipo lograba reconocerla. El error específico que se presentaba era: *"Dispositivo USB desconocido (error de solicitud de descriptor de dispositivo)"* ([Ver anexo, Figura 5](#)). Entre las soluciones

consideradas se incluyó la actualización de los controladores del bus de serie universal, así como la instalación de drivers compatibles tanto con la placa como con el sistema operativo del computador. Sin embargo, estas alternativas no resultaron exitosas.

Otro de los desafíos a los que nos enfrentamos fue la implementación de una función que permitiera calcular la duración, en segundos, durante la ejecución del módulo **Modulo1()**. Para esto se investigaron distintas formas de cuantificar el tiempo transcurrido mientras el Arduino se encontraba en la condición $L \leq L1$. A partir de esta indagación, se identificó el uso de la función **millis()**, la cual retorna el número de milisegundos transcurridos desde el inicio de la ejecución del programa. Mediante la aplicación de esta función, fue posible cumplir con el objetivo propuesto.

Conclusiones

El sistema de detección de luminosidad fue implementado con éxito, logrando medir la intensidad de la luz en el entorno, activar los LEDs y la alarma sonora de acuerdo a los niveles de luminosidad. A través de la implementación del código, también se pudo realizar un seguimiento del tiempo que el sistema estuvo en estado de baja luminosidad.

A través de este proyecto hemos podido entender como el uso de un microcontrolador puede facilitar el diseño de sistemas inteligentes y reactivos en función de las condiciones del ambiente. A pesar de las dificultades técnicas, el sistema cumplió con los objetivos establecidos, proporcionando una solución efectiva para la medición de luminosidad y la automatización de respuestas según diferentes niveles de luz.

En resumen, en este proyecto no solo se refuerza el conocimiento sobre el uso de sensores y microcontroladores, sino también ilustra cómo la programación y la electrónica pueden unirse para resolver problemas cotidianos mediante la automatización y el control inteligente.

Anexos

1. Bloques de código

```
20 int light = 0; // Valor de luminosidad medido por el sensor
21 int r,g,b;
22
23 unsigned long tiempo = 0; // Tiempo de inicio del Modulo1
24 unsigned long duracionEnModulo = 0; // Variable que almacena el tiempo de duración en el Modulo1
25 bool estarEnModulo = false; // Variable bandera para saber si se está en módulo 1
26 bool sonarBuzzer = false; // Variable bandera para evitar repetir la alarma en módulo 4
27
28 // Definición valores de luminosidad (parámetros)
29 int L1 = 500;
30 int L2 = 2900;
31 int L3 = 3800;
```

Bloque de código 1. Variables utilizadas en el programa

```
111
112 void Clasificar(){
113
114     if(light <= L1){
115         if(!estarEnModulo){
116             tiempo = millis(); // Marca el tiempo de entrada a Modulo1 en milisegundos
117             estarEnModulo = true;
118         }
119         duracionEnModulo = millis() - tiempo; // Calcula duración en Modulo1
120
121         printLightBlue(); // Configuración de la pantalla para el Modulo1
122
123         carrier.display.setTextColor(ST77XX_WHITE); // Establecer texto en color blanco
124         carrier.display.setTextSize(2); // Tamaño medio del texto
125         carrier.display.setCursor(20, 160); // Ubicación del texto (x, y)
126
127         // Mostrar en pantalla el tiempo que el módulo ha experimentado el estado light ≤ L1
128
129         carrier.display.print("Tiempo: ");
130         carrier.display.print(duracionEnModulo/1000.0); // Conversión de milisegundos a segundos
131
132         carrier.display.println(" s");
133
134         Modulo1();
135         delay(5000); // Detiene la ejecución del programa durante 5 segundos para visualizar Tiempo en Modulo1
136     }
137
138     else if(light > L1 && light <= L2){
139
140         estarEnModulo = false; // Reinicia variable bandera
141         sonarBuzzer = false; // Reinicia variable bandera
142
143         Modulo2();
144         printLightGreen();
145     }
146     else if(light > L2 && light <= L3){
147
148         estarEnModulo = false; // Reinicia variable bandera
149         sonarBuzzer = false; // Reinicia variable bandera
150
151         Modulo3();
```



```

152     printLightMagenta();
153 }
154 else if(light > L3){
155
156     estarEnModulo = false; // Reinicia variable bandera
157     Modulo4();
158     printLightRed();
159 }
160 }

```

Bloque de código 2. Función Clasificar()

2. Capturas del display



Figura 1. Display módulo 1



Figura 2. Display módulo 2



Figura 3. Display módulo 3

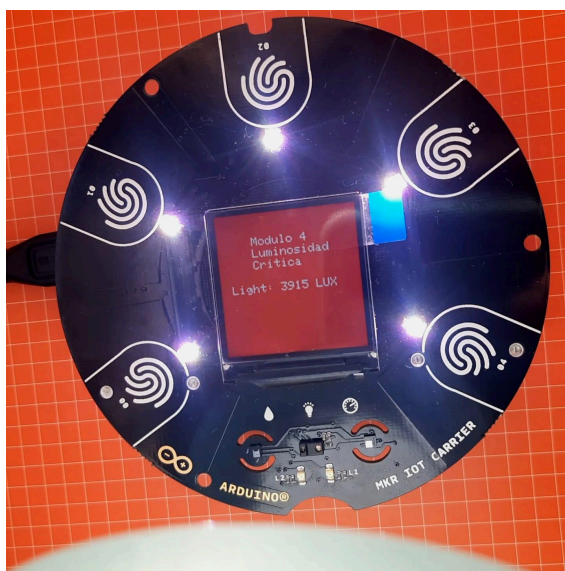


Figura 4. Display módulo 4

3. Errores

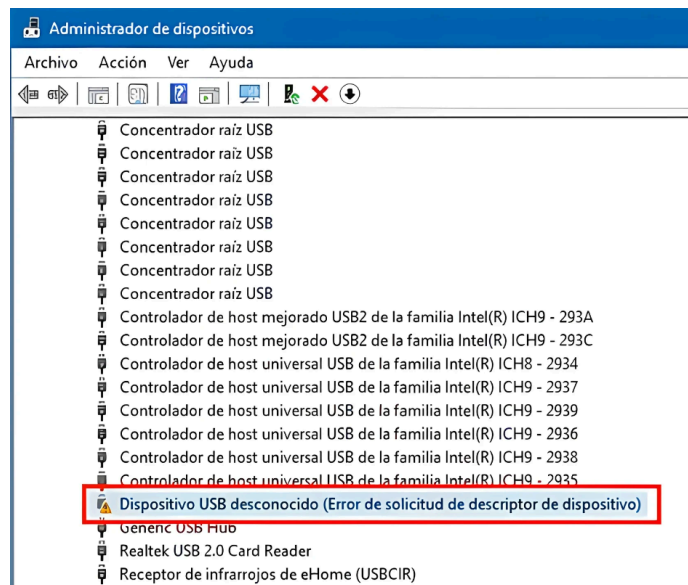


Figura 5. Error dispositivo USB

4. Código fuente

/*

Tarea 3 - Sistemas Digitales - Proyecto n°2 - Medir luminosidad
Este proyecto mide la luminosidad ambiental y clasifica el nivel de luz en 4 rangos, activando LEDs y un buzzer según la condición detectada

Integrantes:

-> Castillo Benjamín
-> Cegarra Ramón
-> Parra Paola
-> Ramos Edinson
-> Rossel Francisco
-> Sanhueza Angelo

*/

```
#include <Arduino_MKRIoTCarrier.h> // Libreria para controlar sensores y actuadores de la placa Arduino MKR IoT Carrier
```

```
MKRIoTCarrier carrier; // Inicializa el objeto de la placa IoT Carrier
```

```
int light = 0; // Valor de luminosidad medido por el sensor  
int r,g,b;
```

```
unsigned long tiempo = 0; // Tiempo de inicio del Modulo1  
unsigned long duracionEnModulo = 0; // Variable que almacena el tiempo de duración en el Modulo1
```

```

bool estarEnModulo = false; // Variable bandera para saber si se está en módulo 1
bool sonarBuzzer = false; // Variable bandera para evitar repetir la alarma en
módulo 4

// Definición valores de luminosidad (parámetros)
int L1 = 500;
int L2 = 2900;
int L3 = 3800;

void setup() {

    Serial.begin(9600); // Prepara a la placa para el envío y recepción de datos
    CARRIER_CASE = false; // Indica que la placa no está dentro del case
    carrier.begin(); // Inicializa los componentes y sensores del Carrier
}

void loop() {

    while (!carrier.Light.colorAvailable()){ // Espera a que se tengan datos desde el
sensor de luz
        delay(5);
    }
    carrier.Light.readColor(r, g, b, light); // Lee valores RGB y nivel de
luminosidad
    Clasificar(); // Evalúa en qué rango está la luminosidad y activa el módulo
correspondiente
}

void Modulo1(){

    // Módulo 1: Luminosidad baja
    // Si light ≤ L1, se mantienen apagados los leds

    carrier.leds.setPixelColor(0, 0, 0, 0); // primer pixel, apagado
    carrier.leds.setPixelColor(1, 0, 0, 0); // segundo pixel, apagado
    carrier.leds.setPixelColor(2, 0, 0, 0); // tercer pixel, apagado
    carrier.leds.setPixelColor(3, 0, 0, 0); // cuarto pixel, apagado
    carrier.leds.setPixelColor(4, 0, 0, 0); // quinto pixel, apagado
    carrier.leds.show(); //actualiza el nuevo estado de los leds
}

void Modulo2(){

    // Módulo 2: Luminosidad media-baja
    // Si L1 < light ≤ L2, se enciende 1 led RGB

    carrier.leds.setPixelColor(0, 255, 0, 0); // primer pixel, rojo
    carrier.leds.setPixelColor(1, 0, 0, 0); // segundo pixel, apagado
    carrier.leds.setPixelColor(2, 0, 0, 0); // tercer pixel, apagado
    carrier.leds.setPixelColor(3, 0, 0, 0); // cuarto pixel, apagado

```

```

    carrier.leds.setPixelColor(4, 0, 0, 0); // quinto pixel, apagado
    carrier.leds.show(); //actualiza el nuevo estado de los leds
}

void Modulo3(){

    // Módulo 3: Luminosidad media-alta
    // Si  $L2 < \text{light} \leq L3$ , se encienden 3 leds RGB

    carrier.leds.setPixelColor(0, 0, 0, 255); // primer pixel, azul
    carrier.leds.setPixelColor(1, 0, 0, 0); // segundo pixel, apagado
    carrier.leds.setPixelColor(2, 0, 0, 255); // tercer pixel, azul
    carrier.leds.setPixelColor(3, 0, 0, 0); // cuarto pixel, apagado
    carrier.leds.setPixelColor(4, 0, 0, 255); // quinto pixel, azul
    carrier.leds.show(); //actualiza el nuevo estado de los leds
}

void Modulo4(){

    // Módulo 4: Luminosidad crítica
    // Si  $\text{light} > L3$ , se encienden los 5 leds RGB y se activa una alarma usando la
    bocina

    carrier.leds.setPixelColor(0, 255, 255, 255); // primer pixel, blanco
    carrier.leds.setPixelColor(1, 255, 255, 255); // segundo pixel, blanco
    carrier.leds.setPixelColor(2, 255, 255, 255); // tercer pixel, blanco
    carrier.leds.setPixelColor(3, 255, 255, 255); // cuarto pixel, blanco
    carrier.leds.setPixelColor(4, 255, 255, 255); // quinto pixel, blanco
    carrier.leds.show(); //actualiza el nuevo estado de los leds

    // Activa la alarma una sola vez mientras se encuentre en el estado  $\text{light} > L3$ 

    if(!sonarBuzzer){
        carrier.Buzzer.sound(500); // Frecuencia establecida en 500 Hz
        delay(3000); // Detiene la ejecución del programa (alarma) durante 3 segundos
        carrier.Buzzer.noSound(); // La alarma se desactiva
        sonarBuzzer = true; // Actualiza el estado
    }
}

// Función para determinar el módulo que se ejecutará

void Clasificar(){

    if(light <= L1){
        if(!estarEnModulo){
            tiempo = millis(); // Marca el tiempo de entrada a Modulo1 en milisegundos
            estarEnModulo = true;
        }
        duracionEnModulo = millis() - tiempo; // Calcula duración en Modulo1
    }
}

```

```

    printLightBlue(); // Configuración de la pantalla para el Modulo1

    carrier.display.setTextColor(ST77XX_WHITE); // Establecer texto en color blanco
    carrier.display.setTextSize(2); // Tamaño medio del texto
    carrier.display.setCursor(20, 160); // Ubicación del texto (x, y)

    // Mostrar en pantalla el tiempo que el módulo ha experimentado el estado light
    ≤ L1

    carrier.display.print("Tiempo: ");
    carrier.display.print(duracionEnModulo/1000.0); // Conversión de milisegundos a
    segundos

    carrier.display.println(" s");

    Modulo1();
    delay(5000); // Detiene la ejecución del programa durante 5 segundos para
    visualizar Tiempo en Modulo1
}

else if(light > L1 && light <= L2){

    estarEnModulo = false; // Reinicia variable bandera
    sonarBuzzer = false; // Reinicia variable bandera

    Modulo2();
    printLightGreen();
}
else if(light > L2 && light <= L3){

    estarEnModulo = false; // Reinicia variable bandera
    sonarBuzzer = false; // Reinicia variable bandera

    Modulo3();
    printLightMagenta();
}
else if(light > L3){

    estarEnModulo = false; // Reinicia variable bandera
    Modulo4();
    printLightRed();
}
}

// Función para establecer configuración de la pantalla para Modulo1

void printLightBlue(){

    carrier.display.fillScreen(ST77XX_BLUE); // Establece fondo azul

```

```

carrier.display.setTextColor(ST77XX_WHITE); // Establece texto en color blanco
carrier.display.setTextSize(2); // Tamaño medio del texto
carrier.display.setCursor(60, 40);
carrier.display.println("Modulo 1");
carrier.display.setCursor(60, 60);
carrier.display.println("Luminosidad");
carrier.display.setCursor(60, 80);
carrier.display.println("Baja");

// Impresión del valor de luminosidad en LUX

carrier.display.setCursor(20, 120); /// Ubicación del texto (x, y)
carrier.display.print("Light: ");
carrier.display.print(light);
carrier.display.println(" LUX");
}

// Función para establecer configuración de la pantalla para Modulo2
void printLightGreen() {

// Ajuste del color de fondo, color del texto, tamaño y posición del texto

carrier.display.fillScreen(ST77XX_GREEN); // Establece fondo verde
carrier.display.setTextColor(ST77XX_WHITE); // Establece texto en color blanco
carrier.display.setTextSize(2); // Tamaño medio del texto
carrier.display.setCursor(60, 40);
carrier.display.println("Modulo 2");
carrier.display.setCursor(60, 60);
carrier.display.println("Luminosidad");
carrier.display.setCursor(60, 80);
carrier.display.println("Media-Baja");

// Impresión del valor de luminosidad en LUX

carrier.display.setCursor(20, 120); // Ubicación del texto (x, y)
carrier.display.print("Light:");
carrier.display.print(light);
carrier.display.println(" LUX");
}

// Función para establecer configuración de la pantalla para Modulo3

void printLightMagenta() {

// Ajuste del color de fondo, color del texto, tamaño y posición del texto

carrier.display.fillScreen(ST77XX_MAGENTA); // Establece fondo magenta
carrier.display.setTextColor(ST77XX_WHITE); // Establece texto en color blanco
carrier.display.setTextSize(2); // Tamaño medio del texto

```

```

carrier.display.setCursor(60, 40);
carrier.display.println("Modulo 3");
carrier.display.setCursor(60, 60);
carrier.display.println("Luminosidad");
carrier.display.setCursor(60, 80);
carrier.display.println("Media-Alta");

// Impresión del valor de luminosidad en LUX

carrier.display.setCursor(20, 120); // Ubicación del texto (x, y)
carrier.display.print("Light: ");
carrier.display.print(light);
carrier.display.println(" LUX");
}

// Función para establecer configuración de la pantalla para Modulo4

void printLightRed() {

    // Ajuste del color de fondo, color del texto, tamaño y posición del texto

    carrier.display.fillScreen(ST77XX_RED); // Establece fondo rojo
    carrier.display.setTextColor(ST77XX_WHITE); // Establece texto en color blanco
    carrier.display.setTextSize(2);
    carrier.display.setCursor(60, 40);
    carrier.display.println("Modulo 4");
    carrier.display.setCursor(60, 60);
    carrier.display.println("Luminosidad");
    carrier.display.setCursor(60, 80);
    carrier.display.println("Critica");

    // Impresión del valor de luminosidad en LUX

    carrier.display.setCursor(20, 120); // Ubicación del texto (x, y)
    carrier.display.print("Light: ");
    carrier.display.print(light);
    carrier.display.println(" LUX");
}

```

Linkografía

1. <https://docs.arduino.cc/>
2. <https://hetpro-store.com/TUTORIALES/arduino-millis/?srsltid=AfmBOorksAl0GbCXLkJnUTDnDCbiwJ5VKqzsi8DzYh3cfErZ2sqcYFAa>
3. <https://robots-argentina.com.ar/didactica/arduino-usando-la-funcion-millis-en-lugar-de-delay/>
4. https://docs.arduino.cc/libraries/arduino_mkriotcarrier/

5. <https://opla.arduino.cc/opla/module/iot-starter-kit-maker-long/lesson/discovering-the-mkr-iot-carrier-long>
6. <https://docs.arduino.cc/language-reference/en/functions/communication/serial/begin/>
7. <https://recoverit.wondershare.es/flashdrive-recovery/flash-drive-not-recognized.html>